Introduction to Computer Architecture
Chapter 2

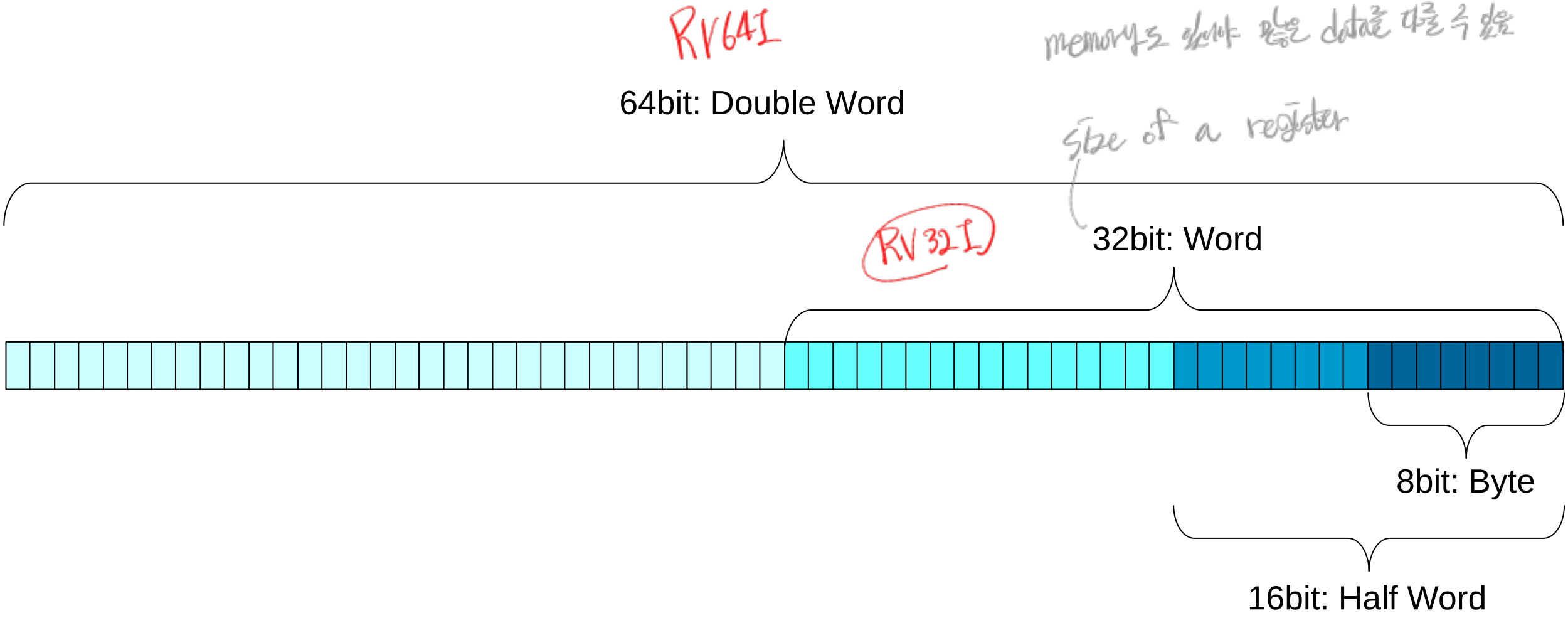
Instructions: Language of the Computer - 2

Hyungmin Cho

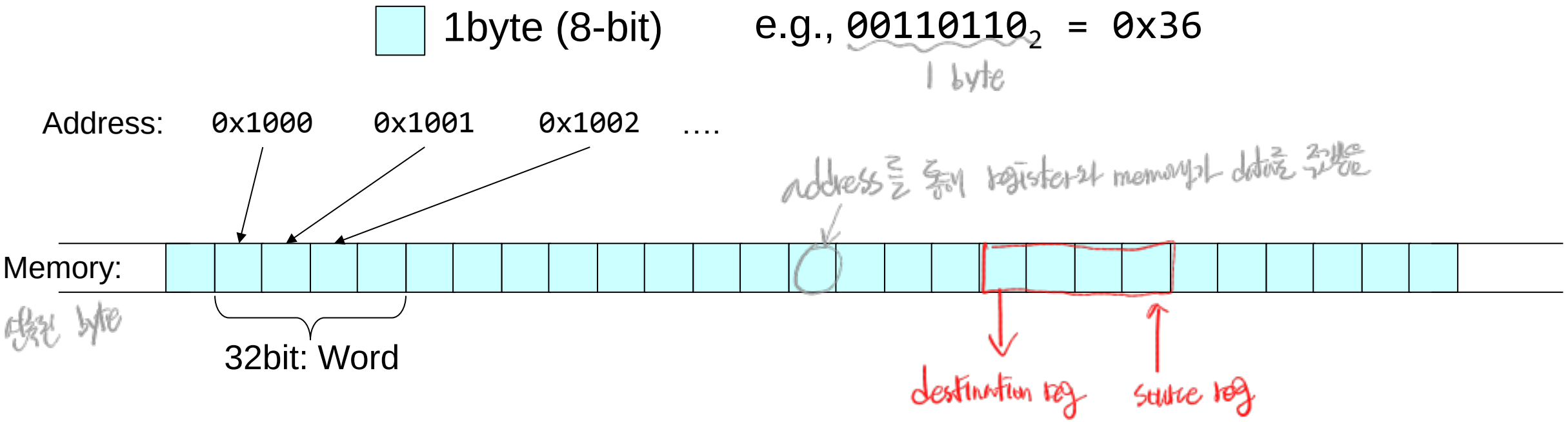
Department of Computer Science and Engineering
Sungkyunkwan University

MEMORY INSTRUCTIONS

Data Size


RV64I
memory로 한번에 받을 수 있는 data의 크기
64bit: Double Word
size of a register
RV32I
32bit: Word
8bit: Byte
16bit: Half Word

Memory Addressing



- Memory "address" is also a 32-bit (non-negative) integer.
  - `0x0000_0000 – 0xFFFF_FFFF`

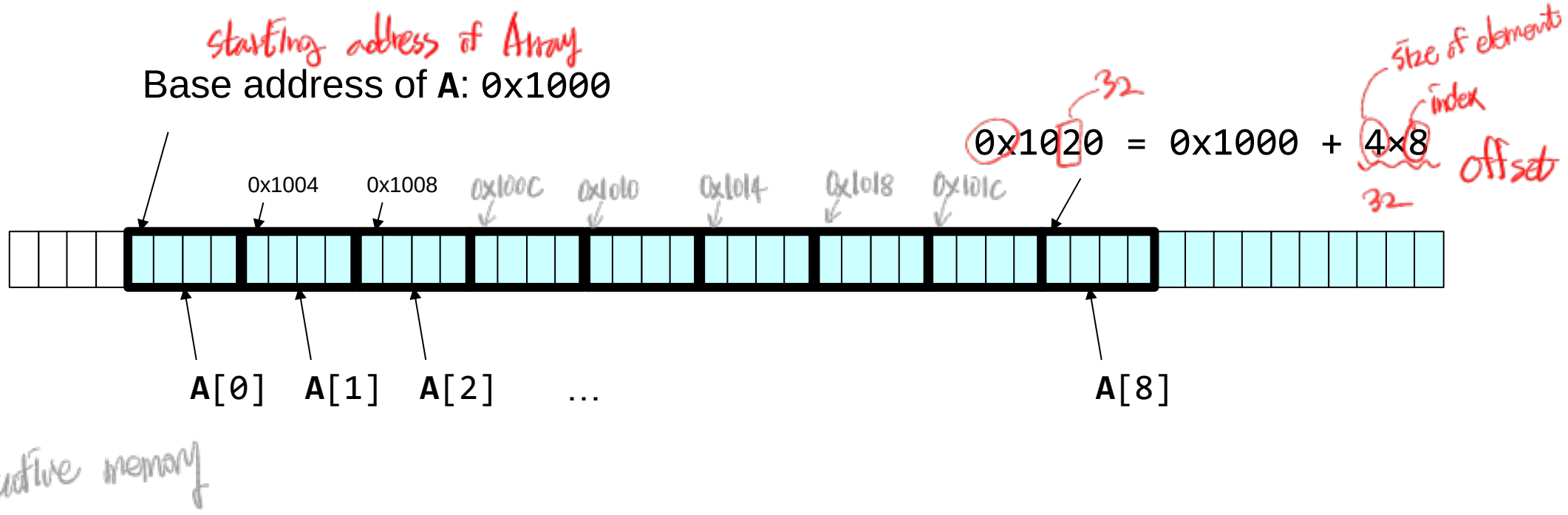
- Memory → register: **Load** Need address & destination register
- Register → memory: **Store** Need address & source register

Memory Operand Example 1

- C code:

```
g = h + A[8];
```

  - g in x1, h in x2, base address (starting address) of **A** in x3
  - **A** is an '`int`' type array (i.e., each element is a 4-byte value)



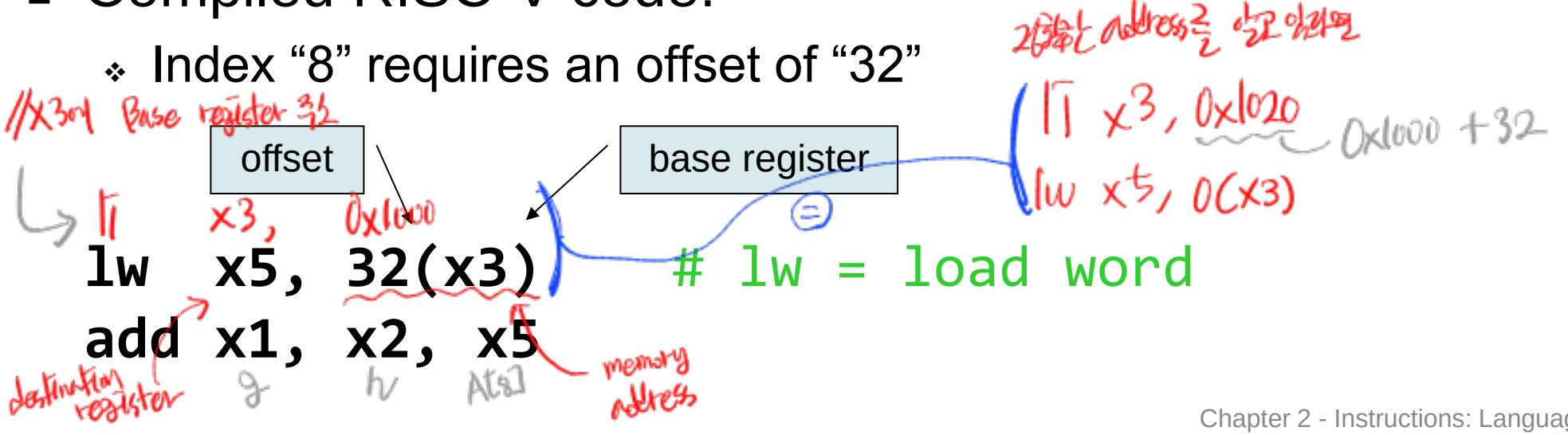
Memory Operand Example 1

- C code:

```
g = h + A[8];
```

  - g in x1, h in x2, base address (starting address) of **A** in x3
  - **A** is an '`int`' type array (i.e., each element is a 4-byte value)

- Compiled RISC-V code:
  - Index "8" requires an offset of "32"



Memory Operand Example 2

- C code:

```
A[12] = h + A[8];
```

  - h in x2, base address of **A** in x3

- Compiled RISC-V code:

```
lw  x6, 32(x3)    # load word
add x7, x2, x6
sw  x7, 48(x3)    # store word
```

Base address가 가리키는 곳에서 offset만큼 떨어진 곳 (다음 Instruction)

```
li  x3, 0x1020
lw  x6, 0(x3)
add x7, x2, x6
li  x3, 0x1030
sw  x7, 0(x3)
```

Memory Operand Example 3

- C code:

```
for( i = 0; i < 10; i++ ) {
    sum = sum + A[i];
}
```

  - `sum` in `x1`, `i` in `x2`, base address of **`A`** in `x3`

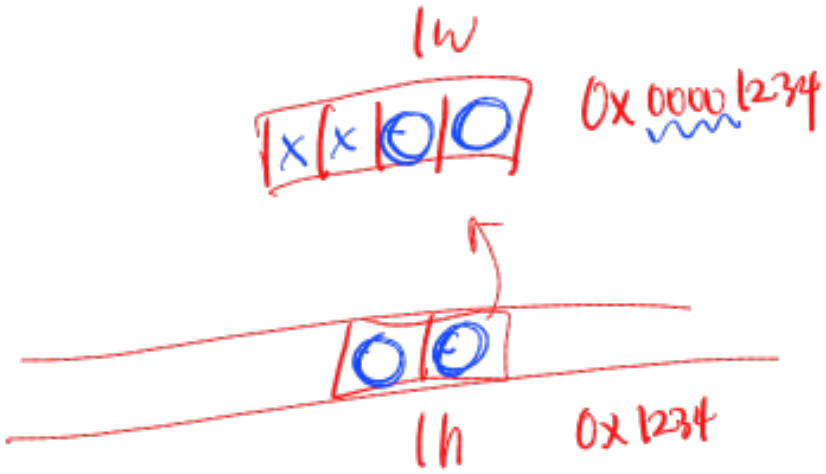
- Compiled RISC-V code (only the loop body):

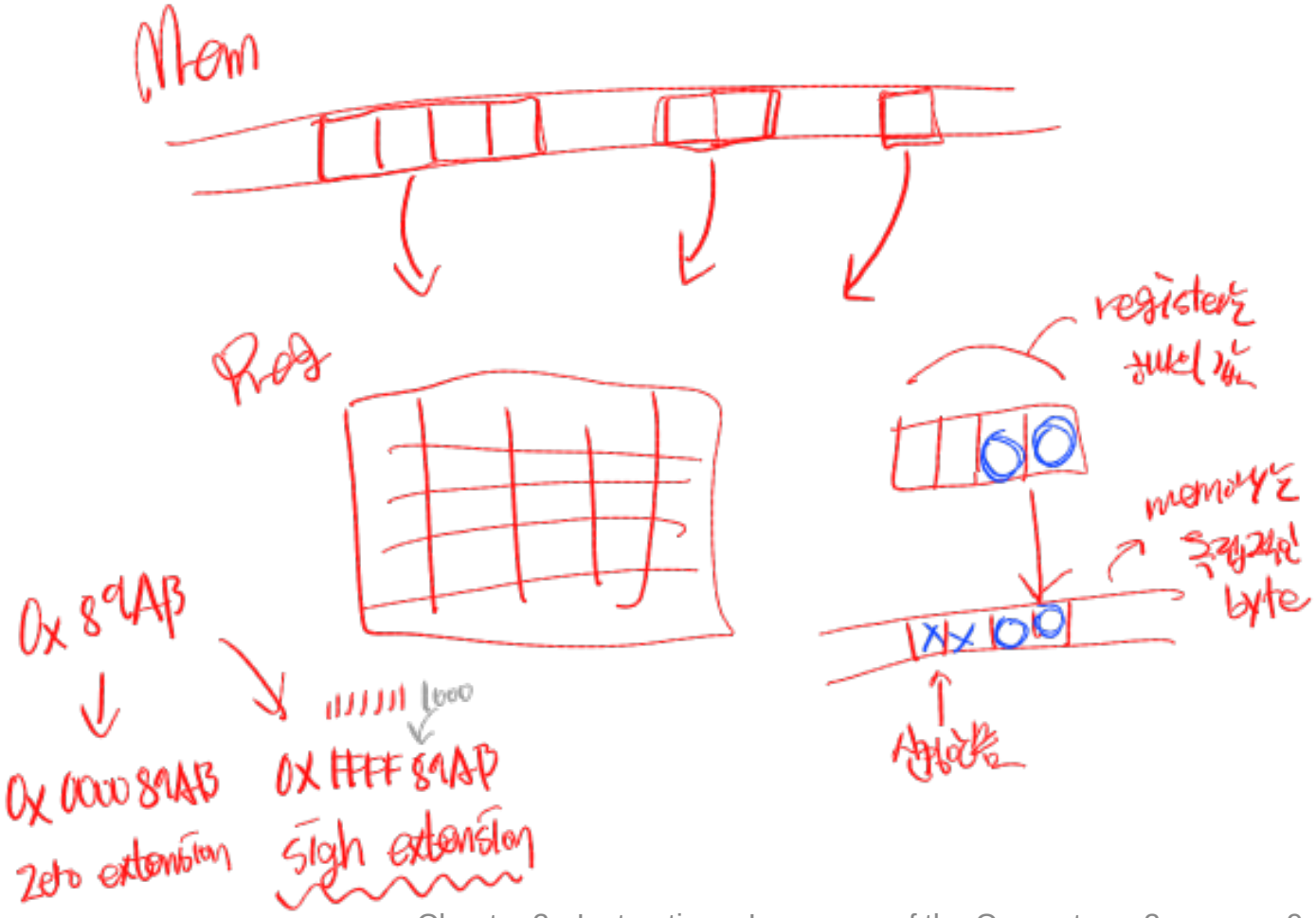
```
…
lw   x5, 0(x3)
add  x1, x1, x5
addi x3, x3, 4
…
```

RISC-V Instructions – Memory

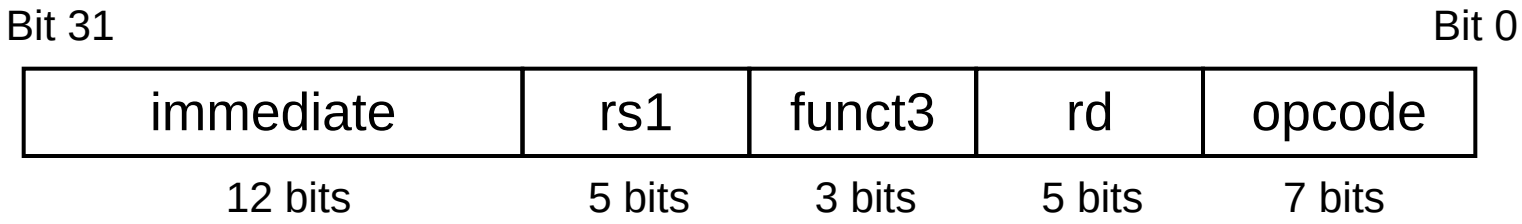
- `lw`, `lh`, `lb`
  - Load word / halfword / byte

- `sw`, `sh`, `sb`
  - Store word / halfword / byte





RISC-V I-format Instructions



opcode rd Imm rs1

```
lw x9,-123(x20)
```

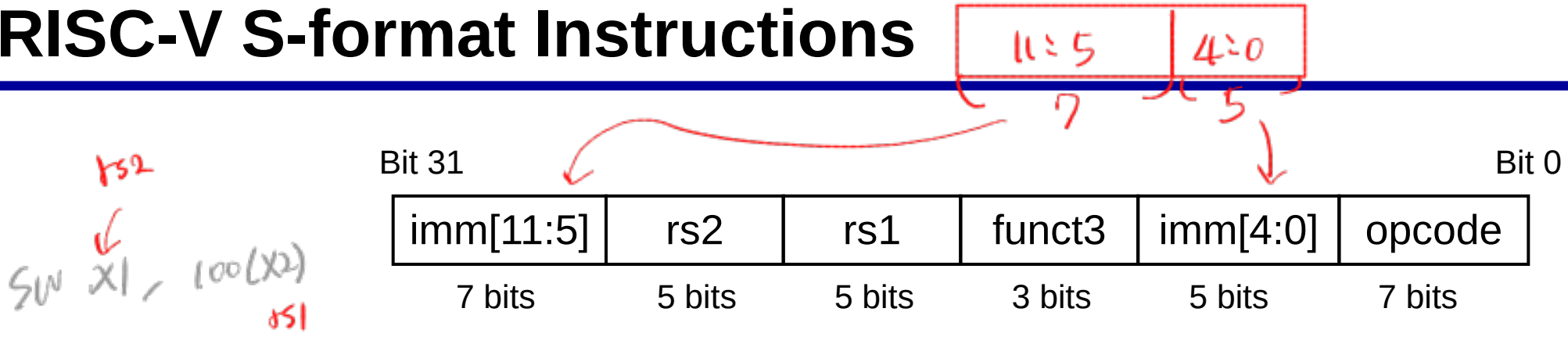
immediate	x20	funct3	x9	Load
-123	20	2	9	3
1111 1000 0101	10100	010	01001	0000011

$$11111000010110100010010010000011_2$$

$$1111\ 1000\ 0101\ 1010\ 0010\ 0100\ 1000\ 0011_2$$

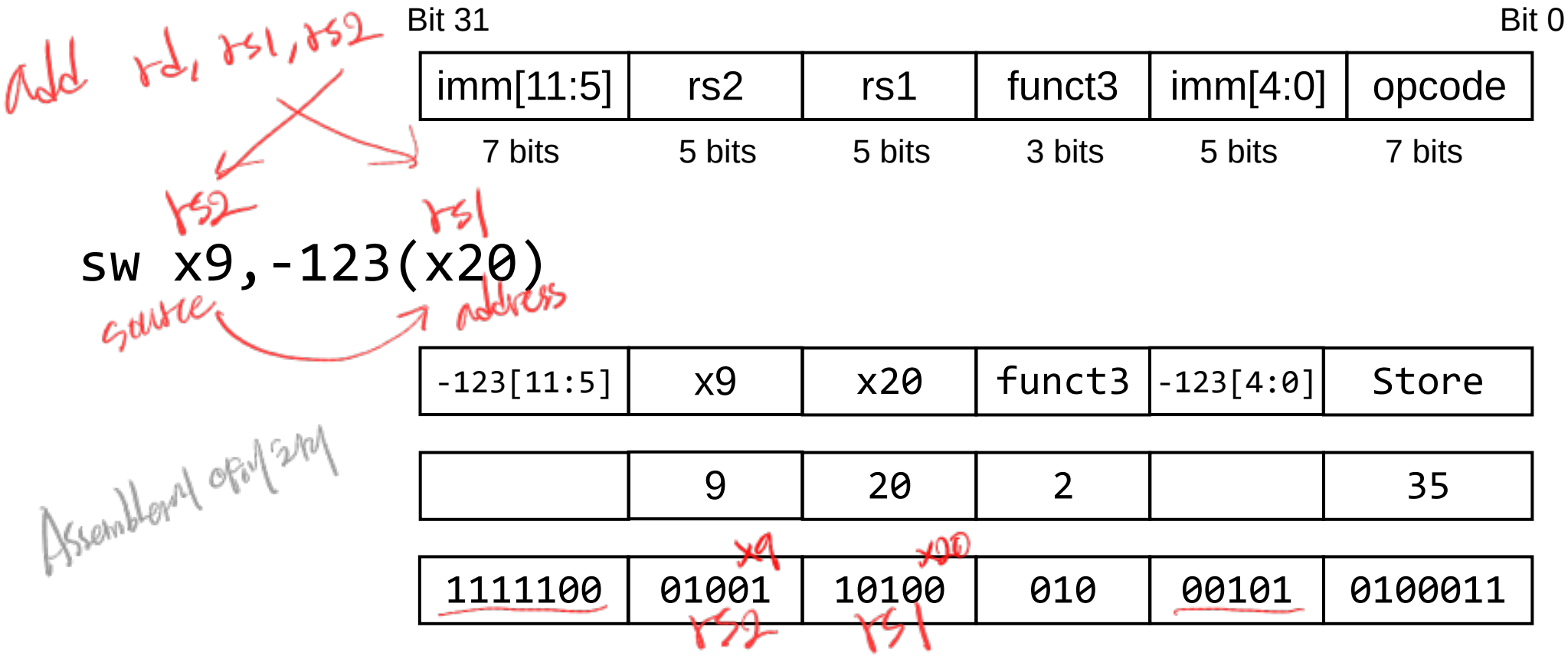
$$= 0xF85A2483$$

RISC-V S-format Instructions



- Store instructions use a different binary format:
  - rs1: base address register number
  - rs2: source operand register number
  - immediate: offset added to base address
    - Designed so that rs1 and rs2 fields are always in the same place

RISC-V S-format Instructions

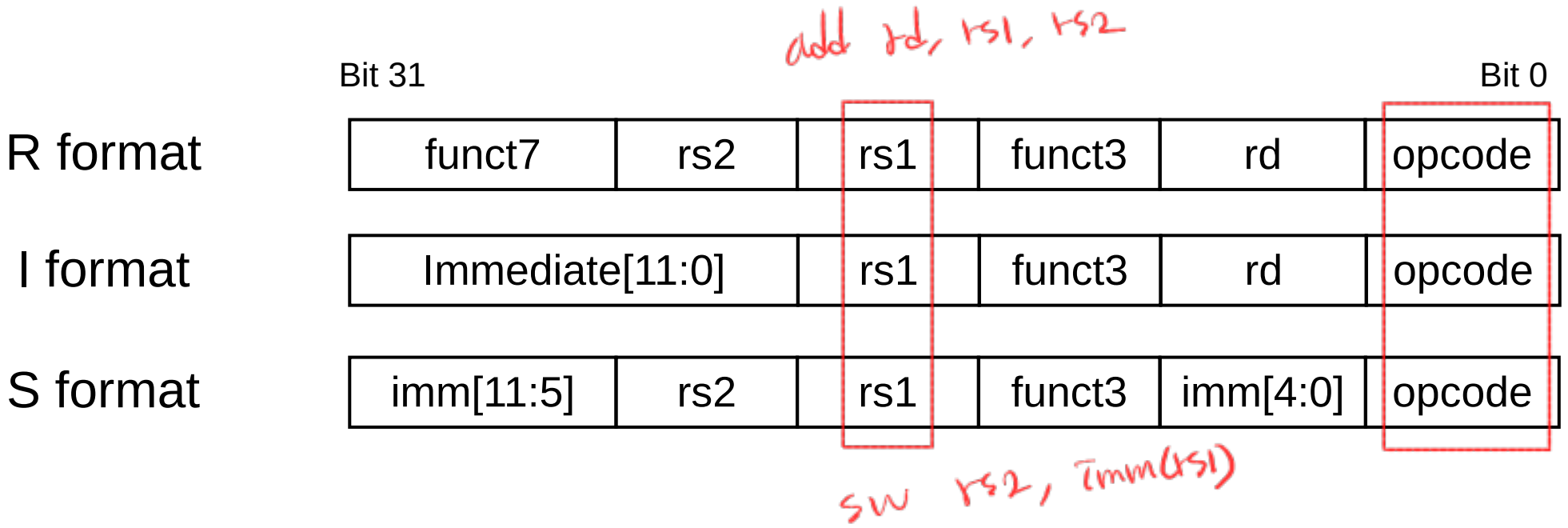


$1111100\ 01001\ 10100\ 010\ 00101\ 0100011_2$

$1111\ 1000\ 1001\ 1010\ 0010\ 0010\ 1010\ 0011_2$

$= 0xF89A22A3$

RISC-V Machine Code Encoding Comparison



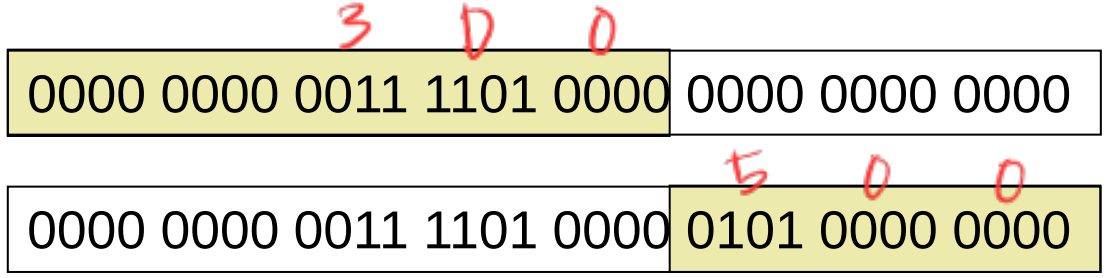
32-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient for common cases
- For occasional 32-bit constant, use **`lui`** (Load upper immediate)

```
lui rd, constant
```

  - Copies a 20-bit constant to [31:12] of `rd`
  - Sets the lower 12 bits of `rd` to 0
  - e.g., **`lui x10, 0x123`** → `x10` becomes `0x00123000`

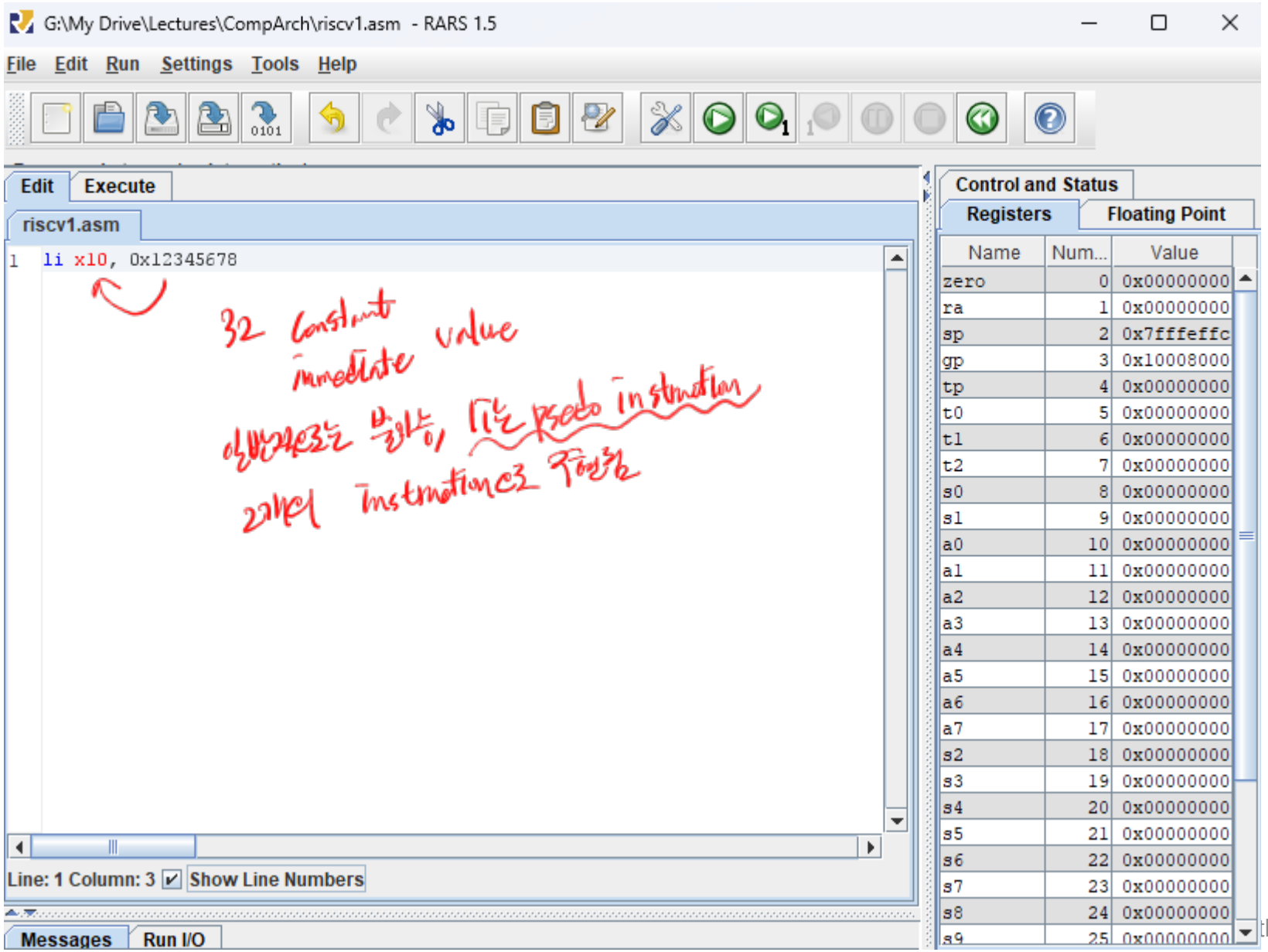
```
lui x19, 976 //0x003D0
addi x19, x19, 1280 //0x500
```



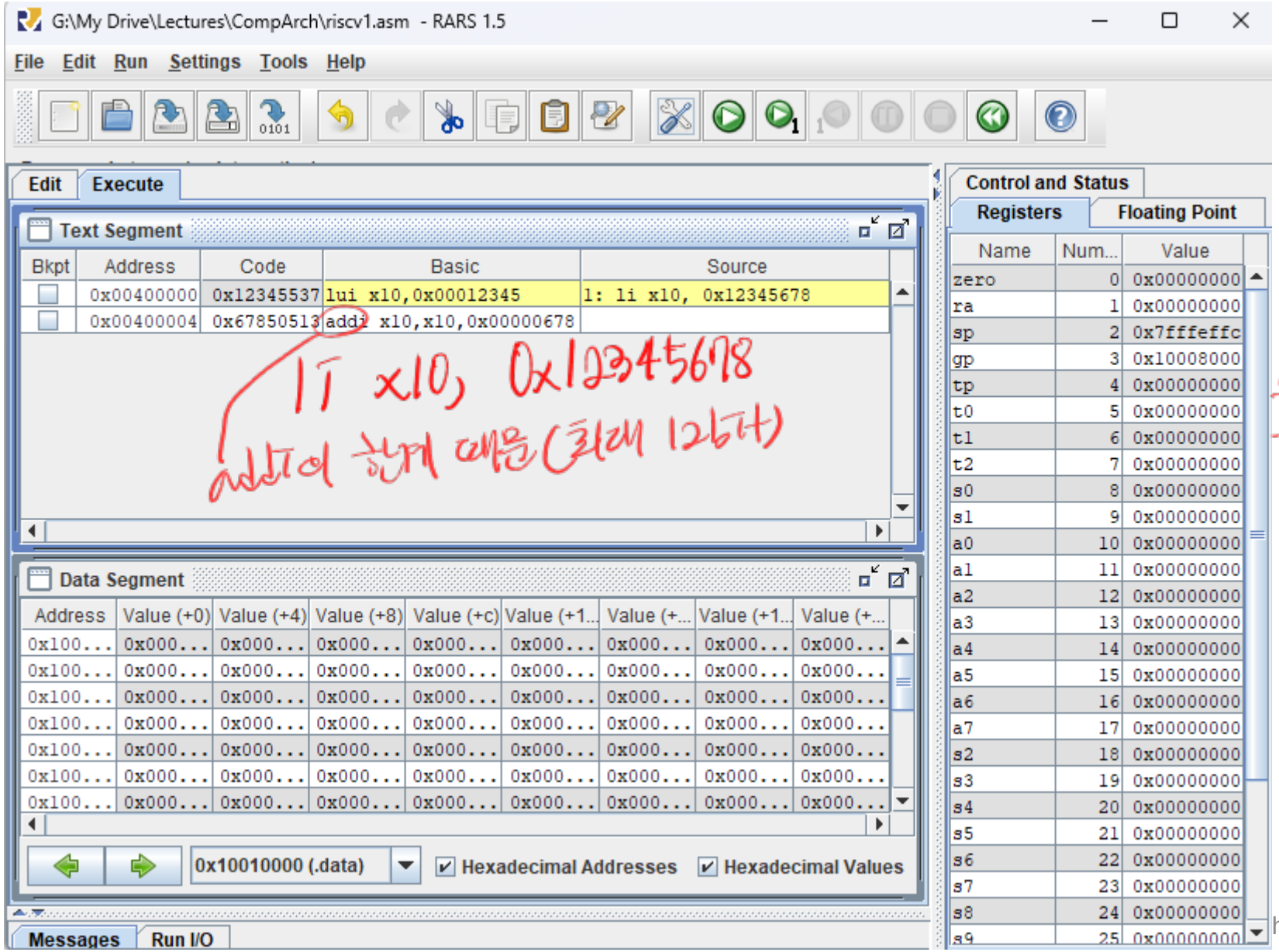
x19 : 0x3D0500

LUI Example

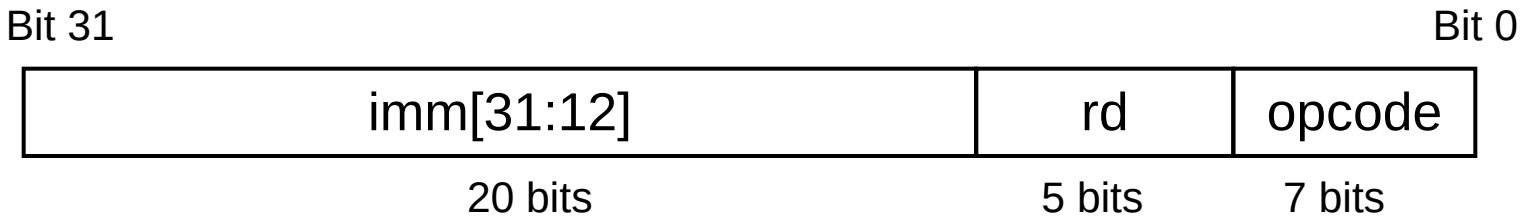
RARS RISC-V Simulator



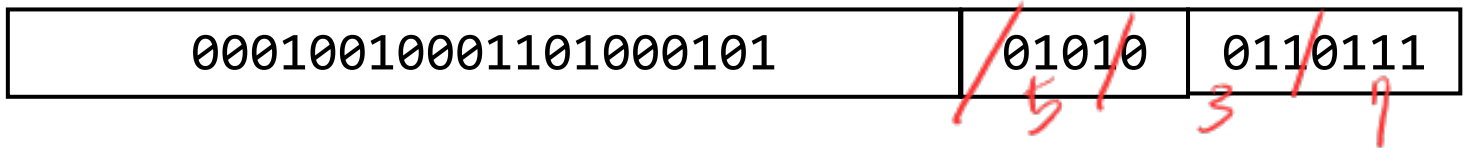
LUI Example


G:\My Drive\Lectures\CompArch\riscv1.asm - RARS 1.5
File Edit Run Settings Tools Help
Edit Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x12345537 lui x10,0x00012345 1: li x10, 0x12345678
0x00400004 0x67850513 addi x10,x10,0x00000678
Data Segment
Address Value (+0) Value (+4) Value (+8) Value (+c) Value (+1... Value (+... Value (+1... Value (+...
0x10010000 (.data)
Hexadecimal Addresses
Hexadecimal Values
Messages Run I/O
Control and Status
Registers Floating Point
Name Num... Value
zero 0 0x00000000
ra 1 0x00000000
sp 2 0x7fffeffc
gp 3 0x10008000
tp 4 0x00000000
t0 5 0x00000000
t1 6 0x00000000
t2 7 0x00000000
s0 8 0x00000000
s1 9 0x00000000
a0 10 0x00000000
a1 11 0x00000000
a2 12 0x00000000
a3 13 0x00000000
a4 14 0x00000000
a5 15 0x00000000
a6 16 0x00000000
a7 17 0x00000000
s2 18 0x00000000
s3 19 0x00000000
s4 20 0x00000000
s5 21 0x00000000
s6 22 0x00000000
s7 23 0x00000000
s8 24 0x00000000
s9 25 0x00000000
li x10, 0x12345678
addi 하위 비트 (12bit)
0x12345000
+ 0x678

LUI Binary Encoding – U format




```
lui x10, 0x12345
```



$00010010001101000101\ 01010\ 0110111_2$

$0001\ 0010\ 0011\ 0100\ 0101\ 0101\ 0011\ 0111_2$

$= 0x12345537$

Sign Extension

- RISC-V instructions sign-extend the immediate value

```
li t0, 11
addi t1, t0, -10
```

- **`lh`** and **`lb`** also sign-extends the loaded halfword or byte

- **`lhu`** and **`lbu`** zero-extends the loaded halfword or byte

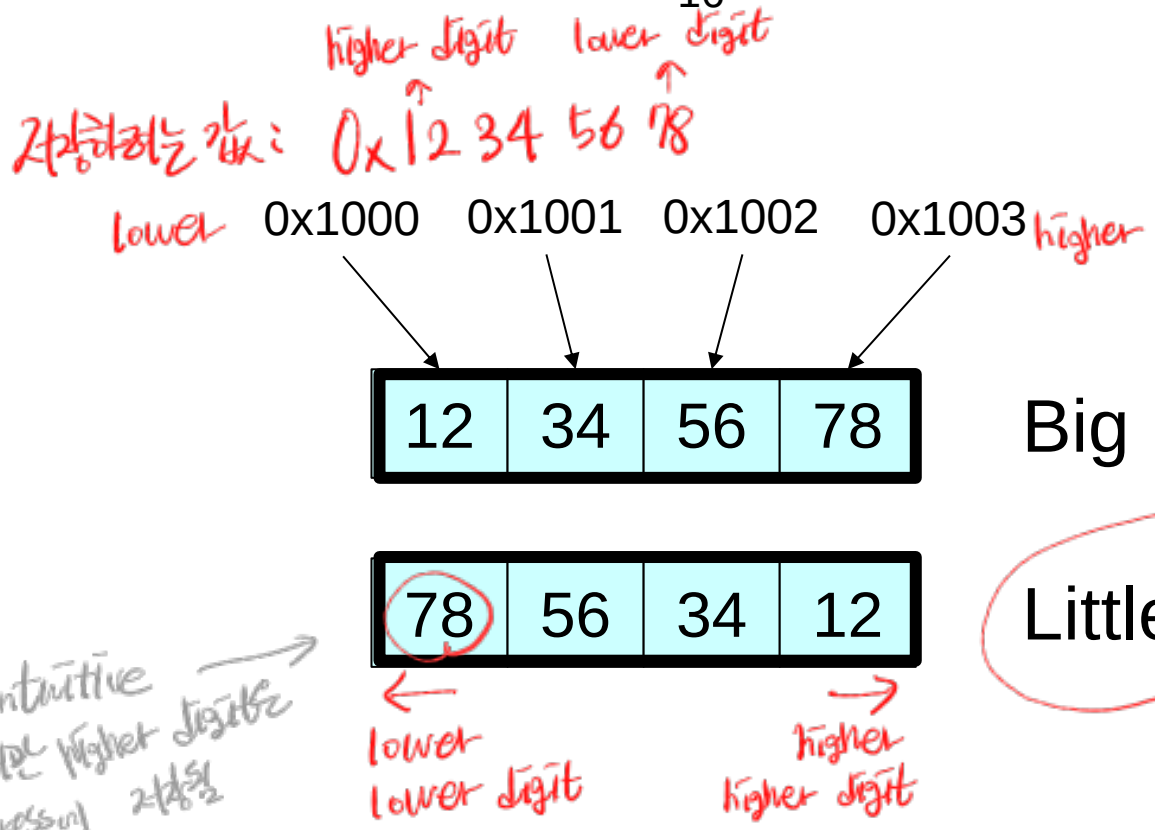
unsigned

- There are no **`shu`** or **`sbu`** instructions!

Endianness

What happens if store this 4-byte value to address 0x1000 ?

$$\mathbf{12\ 34\ 56\ 78}_{16} = 1\times16^7 + 2\times16^6 + 3\times16^5 \ldots + 7\times16^1 + 8\times16^0$$



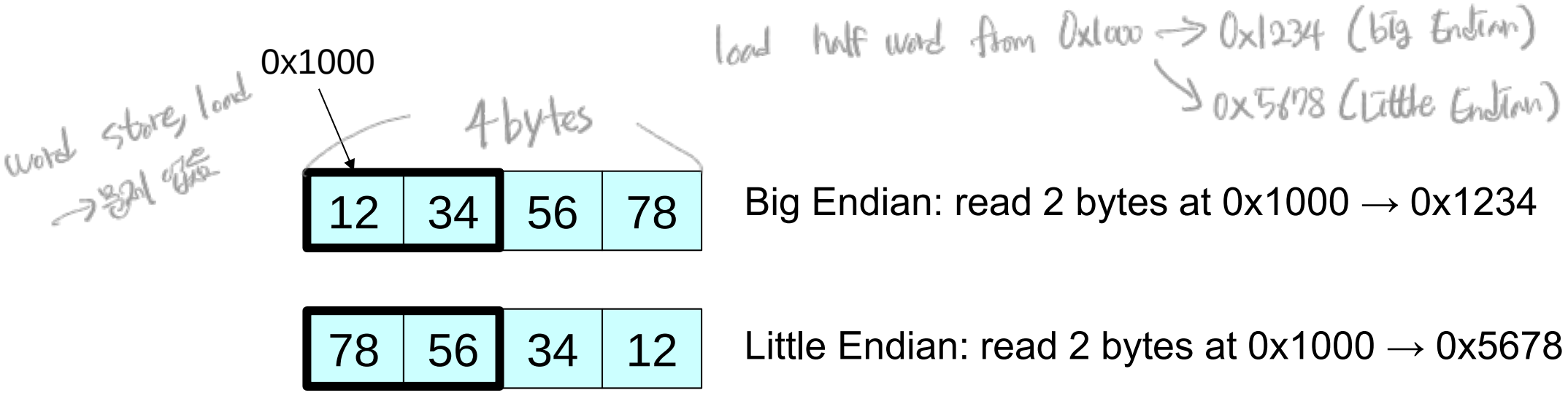
Big Endian : MIPS, SPARC

Little Endian : x86, **RISC-V**

Endianness

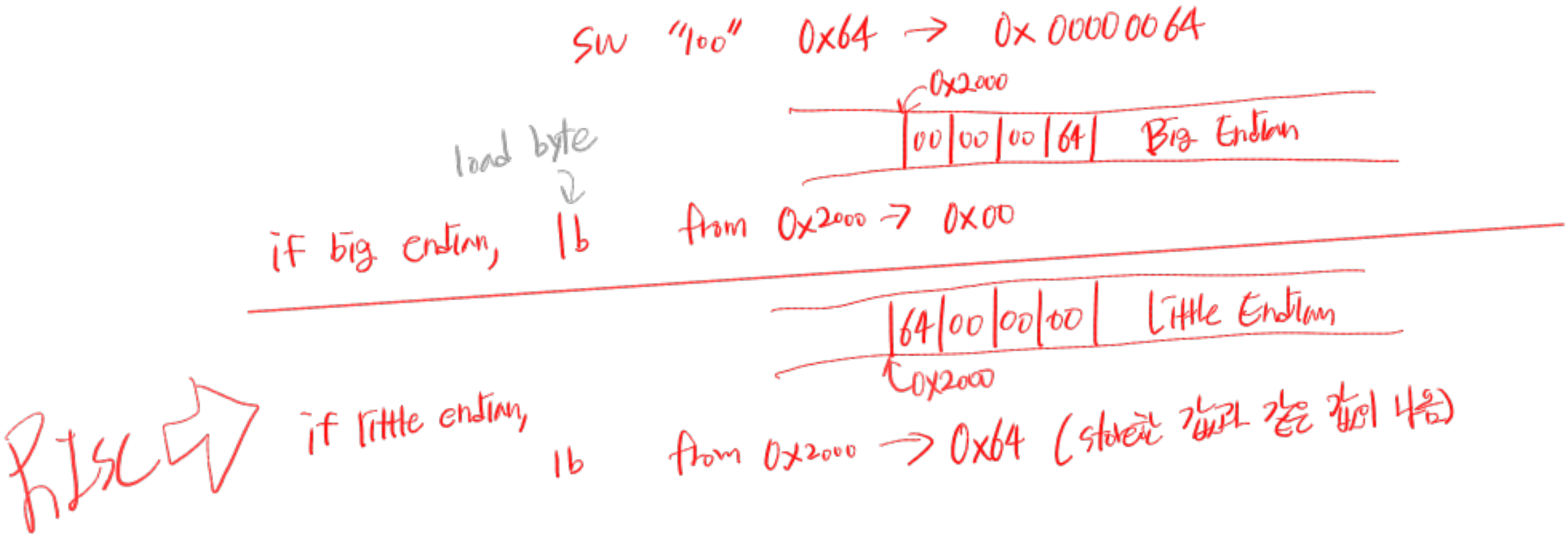
- Endianness does not matter when storing/loading 4 bytes at a time at 4-byte aligned addresses.

- It matters when you read in smaller sizes.



Big Endian: read 2 bytes at 0x1000 → 0x1234

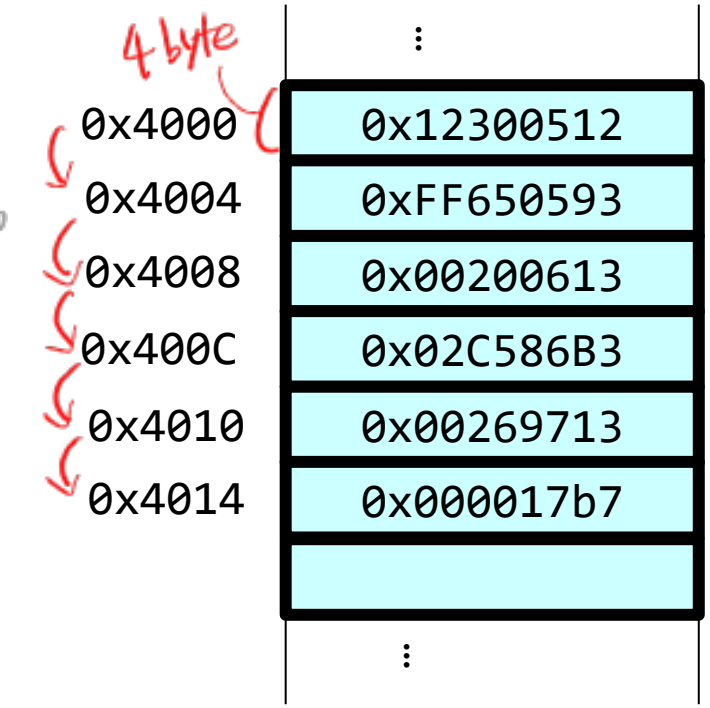
Little Endian: read 2 bytes at 0x1000 → 0x5678



CONTROL FLOW INSTRUCTIONS

Program Counter

- Program Counter (PC)
  - Indicates the memory address of the current instruction



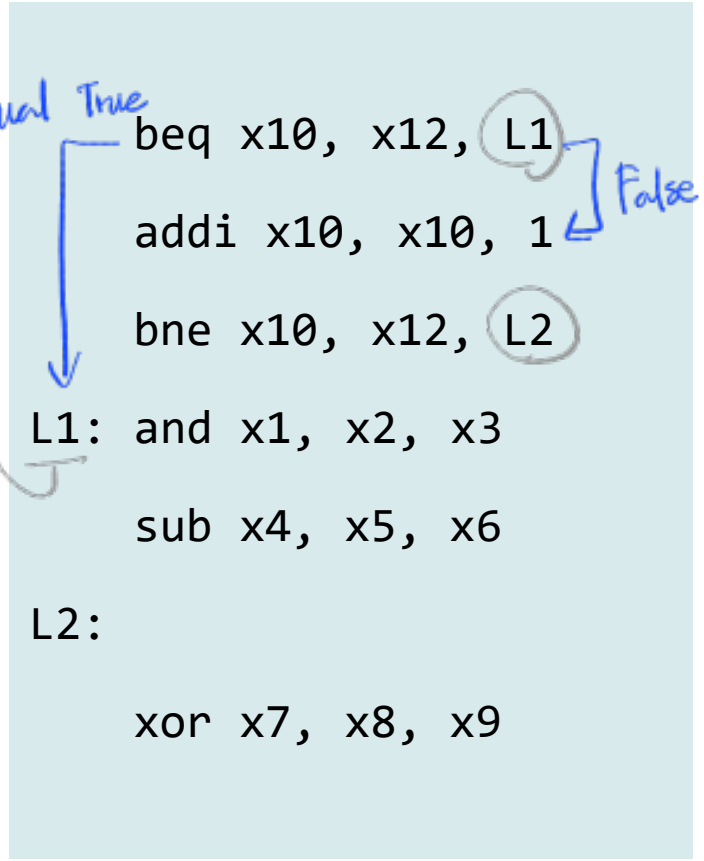

```
addi x10, x0, 0x123
addi x11, x10, -10
addi x12, x0, 2
mul  x13, x11, x12
slli x14, x13, 2
lui  x15, 1
```

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue to the next instruction..

- **`beq rs1, rs2, L1`**
  - `if(*rs1 == *rs2)`, jump to the instruction at label `L1`

- **`bne rs1, rs2, L2`**
  - `if(*rs1 != *rs2)`, jump to the instruction at label `L2`



More Conditional Operations

branch less than

- **`blt rs1, rs2, L3`**
  - `if(*rs1 < *rs2)`, jump to the instruction at label `L3`

branch greater equal

- **`bge rs1, rs2, L4`**
  - `if(*rs1 >= *rs2)`, jump to the instruction at label `L4`

rs1 값 < rs2보다 작다
≥ rs2보다 크거나 같다

Signed vs. Unsigned

- Signed comparison: **`blt`**, **`bge`**
- Unsigned comparison: **`bltu`**, **`bgeu`**

- Example
  - `x22 = 1111 1111 1111 1111 1111 1111 1111 1111`
  - `x23 = 0000 0000 0000 0000 0000 0000 0000 0001`
  - `x22 < x23 // signed comparison`
    - `-1 < +1`
  - `x22 > x23 // unsigned comparison`
    - `+4,294,967,295 > +1`

Compiling If Statements

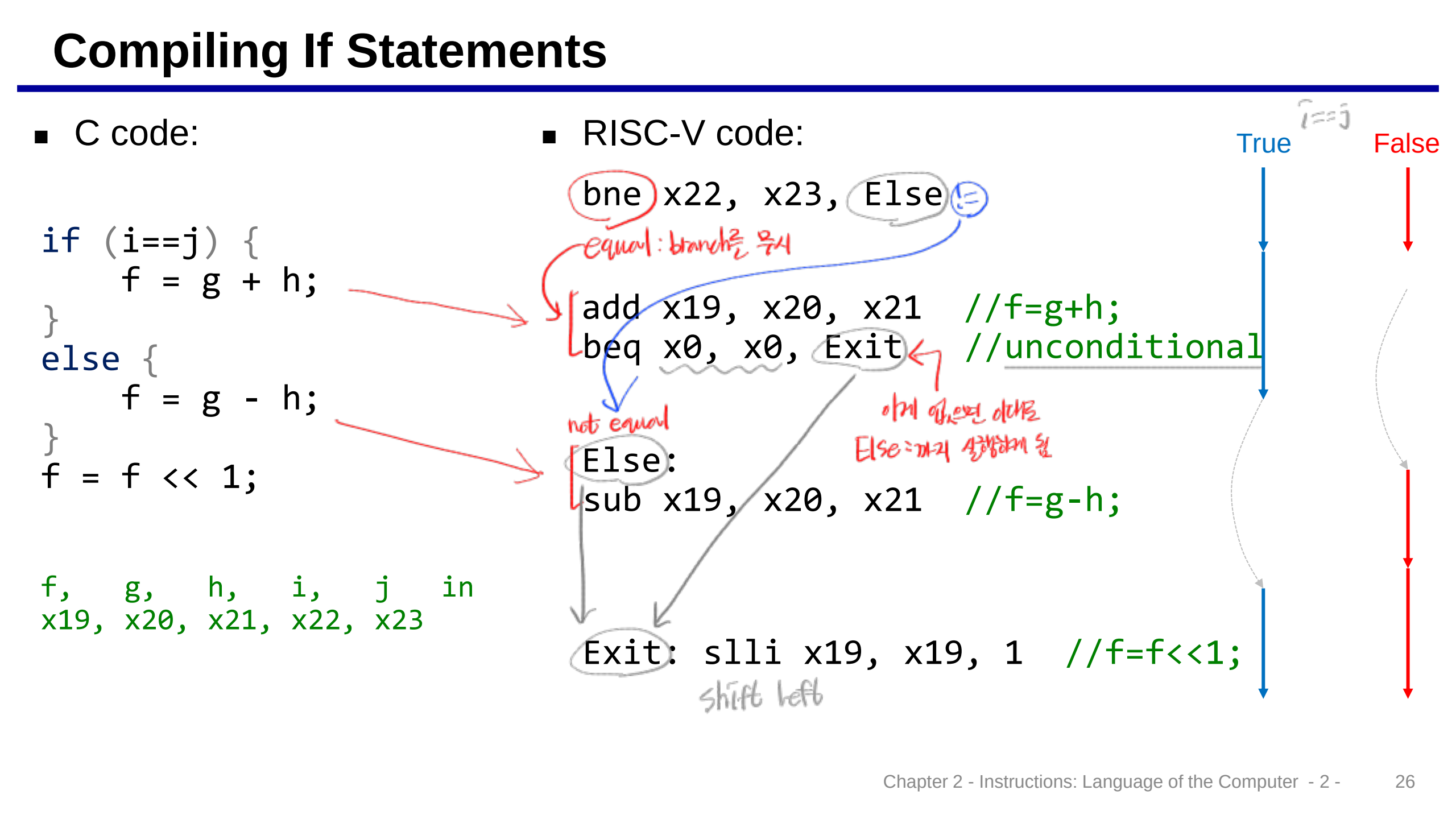
- C code:

```
if (i==j) {
    f = g + h;
}
else {
    f = g - h;
}
f = f << 1;
```

```
f,   g,   h,   i,   j   in
x19, x20, x21, x22, x23
```

- RISC-V code:

```
    bne x22, x23, Else
    add x19, x20, x21   //f=g+h;
    beq x0, x0, Exit    //unconditional
Else:
    sub x19, x20, x21   //f=g-h;
Exit: slli x19, x19, 1  //f=f<<1;
```



Compiling If Statements

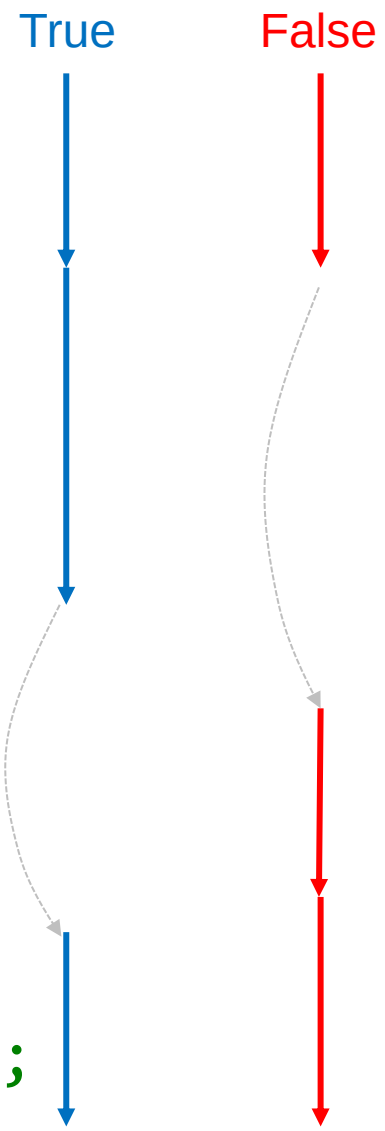
- C code:

```
if (i<10) {
    f = g + h;
}
else {
    f = g - h;
}
f = f << 1;
```

f, g, h, i, j in x19, x20, x21, x22, x23

- RISC-V code:

```
    addi x10, x0, 10
    bge x22, x10, Else
    add x19, x20, x21  //f=g+h;
    j   Exit
Else:
    sub x19, x20, x21  //f=g-h;
Exit: slli x19, x19, 1  //f=f<<1;
```



Compiling Loop Statements

- C code:

```
k = 0;
while (i>=0) {
    i += save[k];
    k++;
}
```

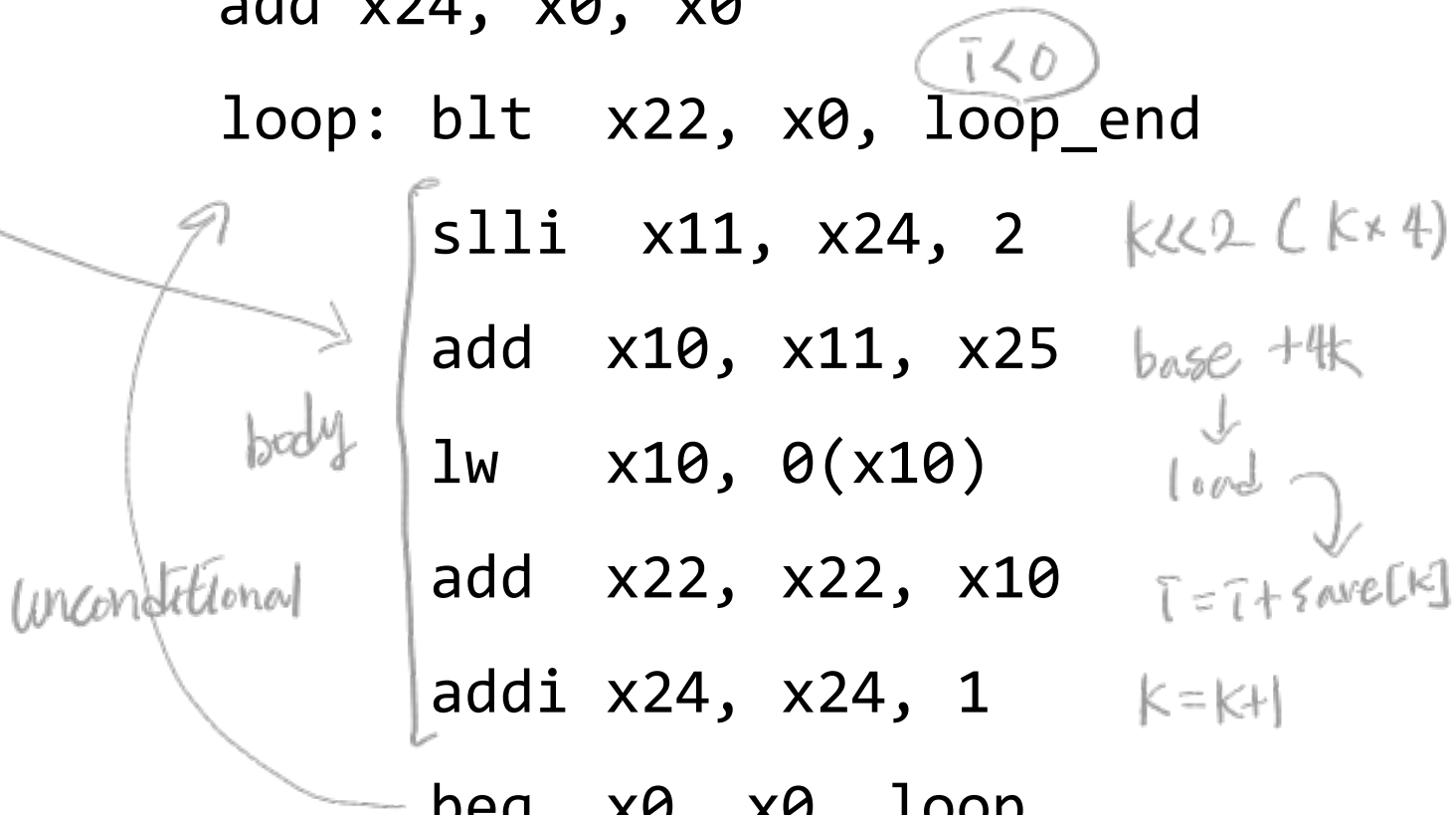
i in x22, k in x24

save is an **int** array

base address of save in x25

- RISC-V code:

```
    add x24, x0, x0
loop: blt  x22, x0, loop_end
    slli  x11, x24, 2
    add   x10, x11, x25
    lw    x10, 0(x10)
    add   x22, x22, x10
    addi x24, x24, 1
    beq   x0, x0, loop
loop_end: …
```



Compiling Loop Statements

- C code:

```
k = 0;
while (i>=0) {
      i += save[k];
      k++;
}
```

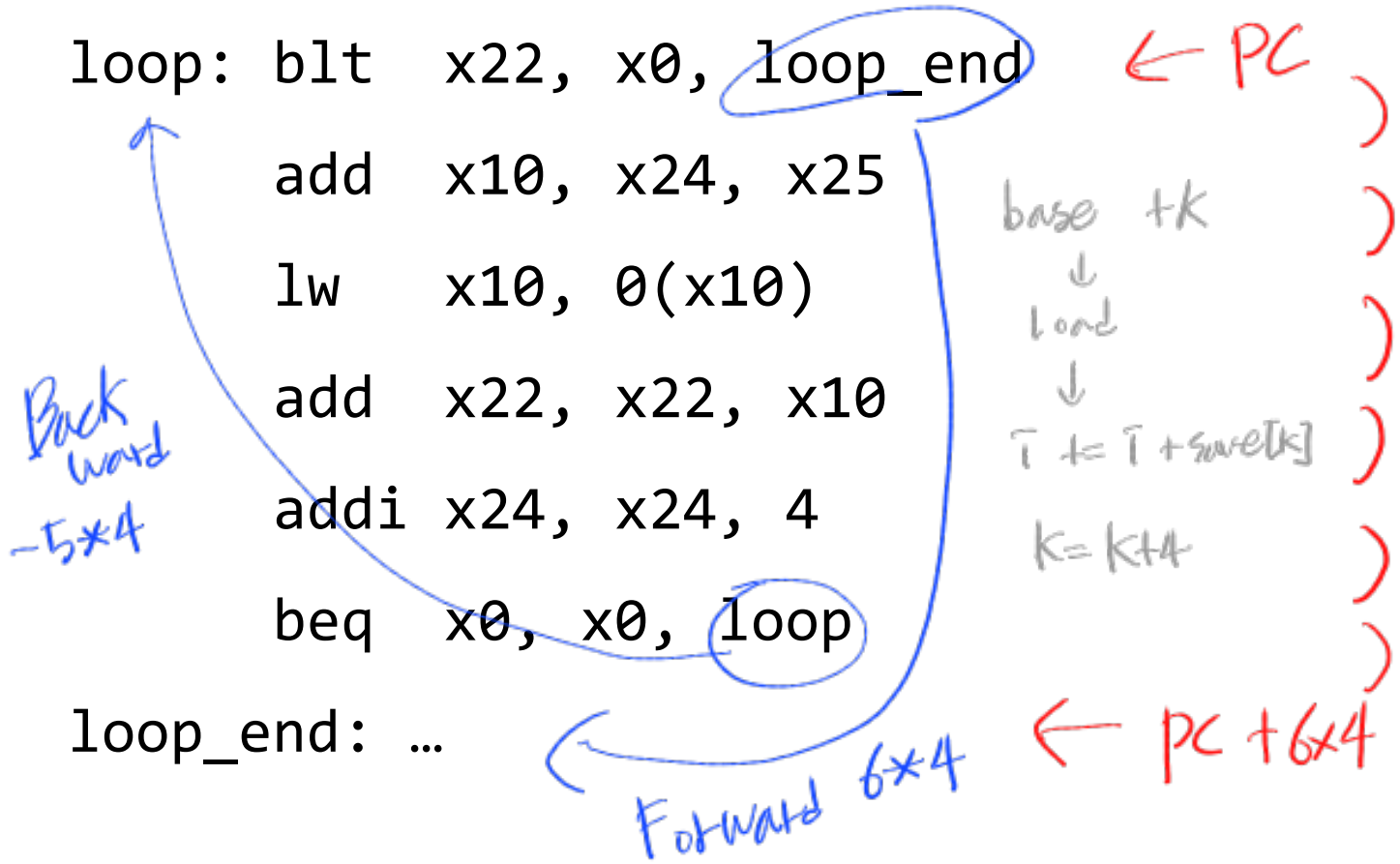
i in x22, k*4 in x24

save is an **int** array

base address of save in x25

- RISC-V code:

```
add x24, x0, x0
loop: blt  x22, x0, loop_end
      add  x10, x24, x25
      lw   x10, 0(x10)
      add  x22, x22, x10
      addi x24, x24, 4
      beq  x0, x0, loop
loop_end: …
```



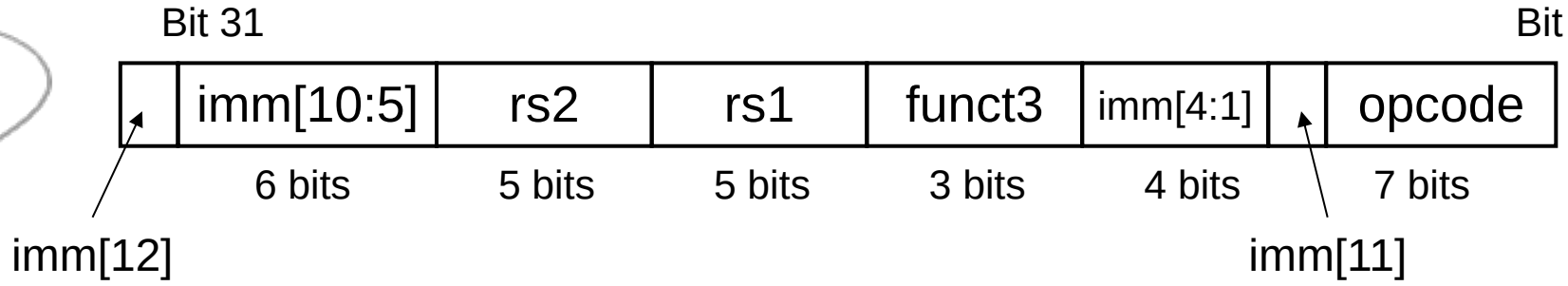
Branch Addressing

- Branch instructions specify:
  - Opcode, two registers, branch target
- PC-relative addressing
  - Target address = PC + immediate
  - Forward or backward
- Most branch targets are near the branch instruction
  - Immediate is a 13-bit signed value, and imm[0] is assumed to be zero.

Immediate
13bit → 0000 ... 1001
actual offset
0000 ... 1010
13bit

program counter

SB format



Branch Encoding Example

beq : SB format

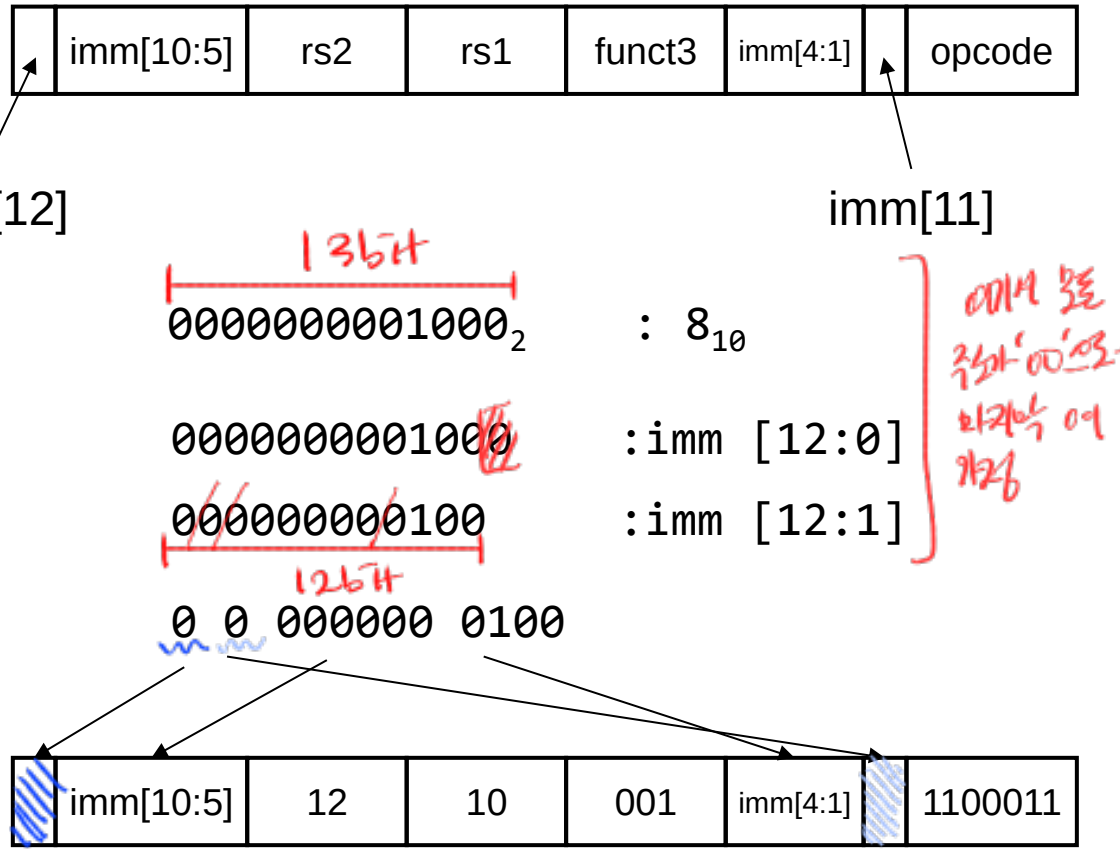
Address: 0x80002000

```
    bne x10, x12, TGT
    xor x4, x5, x6
TGT: add x1, x2, x3
```

+0x8

+8

Address: 0x80002008

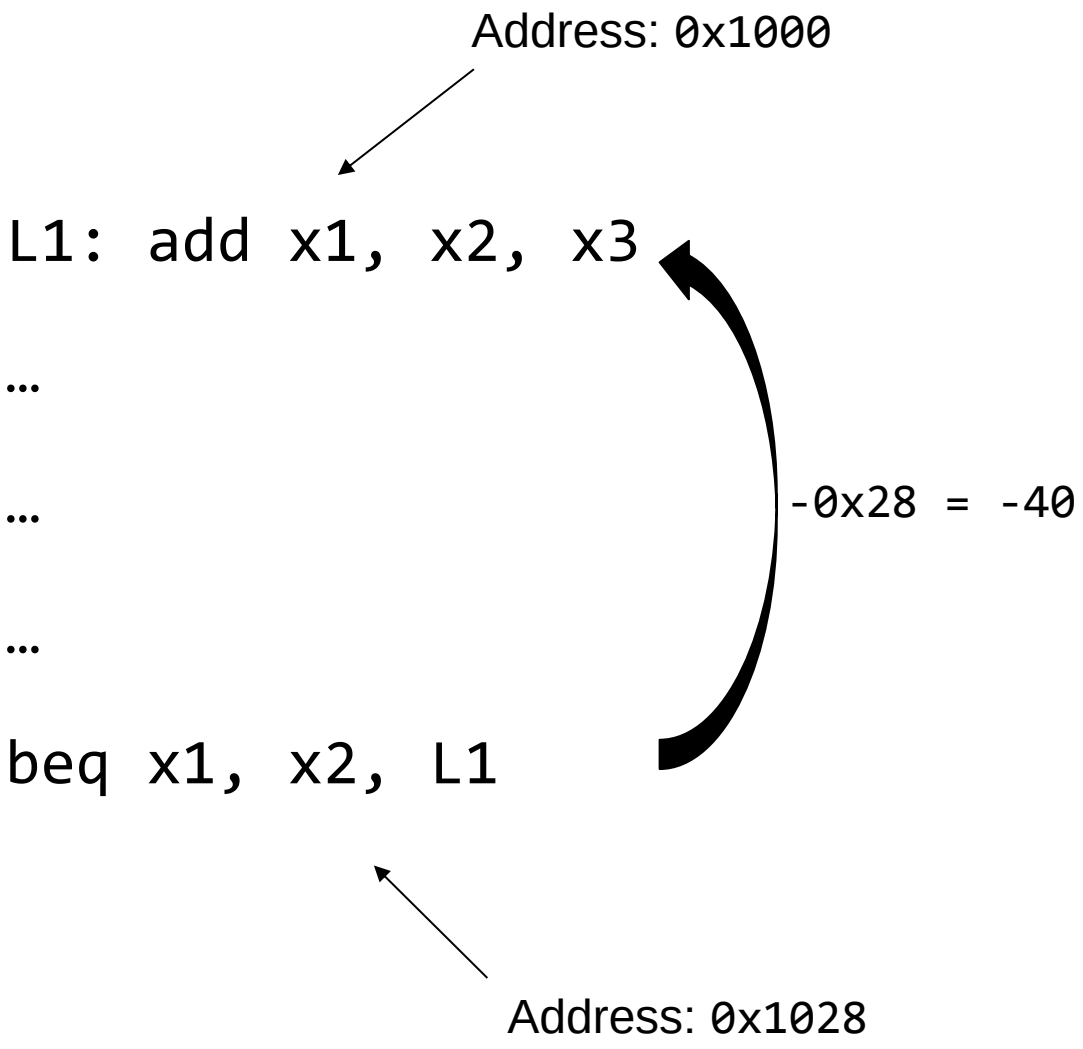


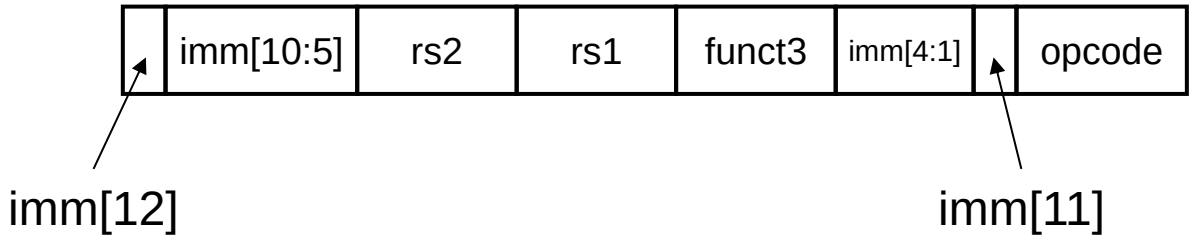
$0\ 000000\ 01100\ 01010\ 001\ 0100\ 0\ 1100011_2$

$0000\ 0000\ 1100\ 0101\ 0001\ 0100\ 0110\ 0011_2$

$= 0x00C51463$

Branch Encoding Example



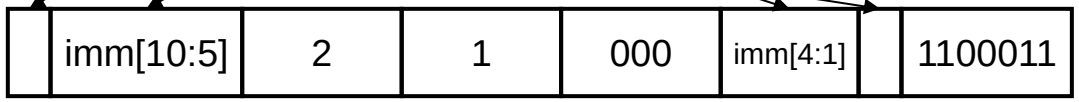


1111111011000_2 : -40_{10}

1111111011000 :imm [12:0]

1111111 01100 :imm [12:1]

1 1 111110 1100



$1\ 111110\ 00010\ 00001\ 000\ 1100\ 1\ 1100011_2$

$1111\ 1100\ 0010\ 0000\ 1000\ 1100\ 1110\ 0011_2$

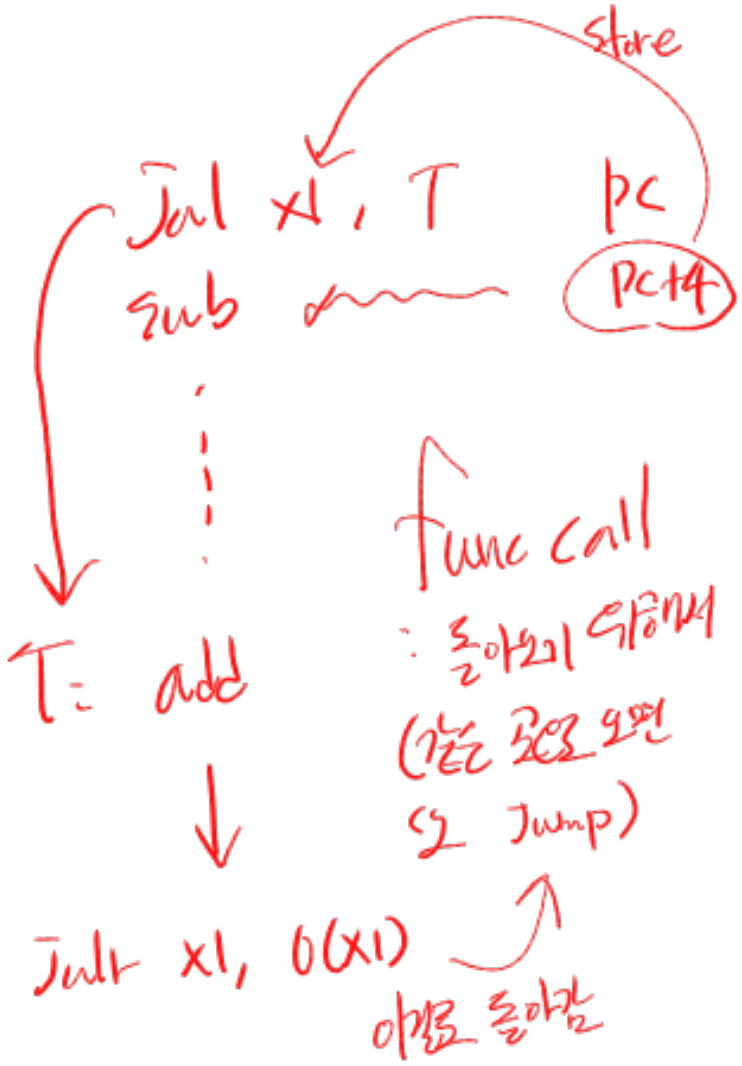
= 0xFC208CE3

RISC-V Jump Instructions

- **`jal x1, L1 :`** jump-and-link
  - Jump to L1
  - Address of the following instruction (PC+4) → x1

- **`jalr x1, offset(x2) :`** jump register
  - Jump to offset + address in x2
  - Address of the following instruction (PC+4) → x1

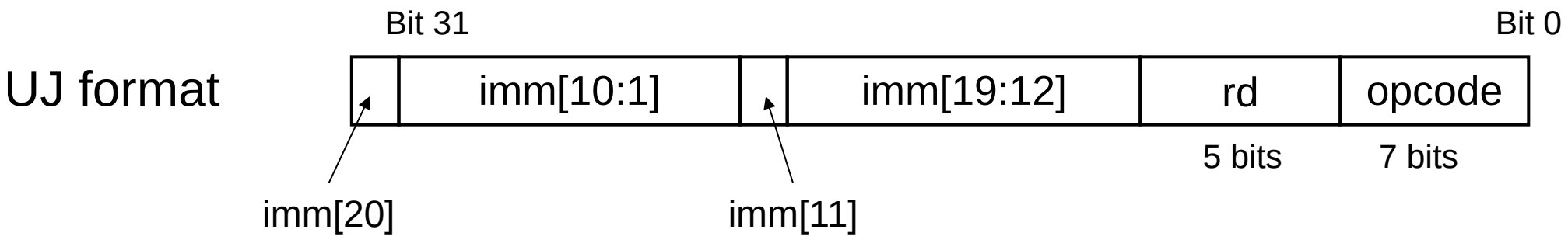
- **`j L1 :`** jump
  - Pseudo-instruction
  - `jal x0, L1`



Jump Addressing

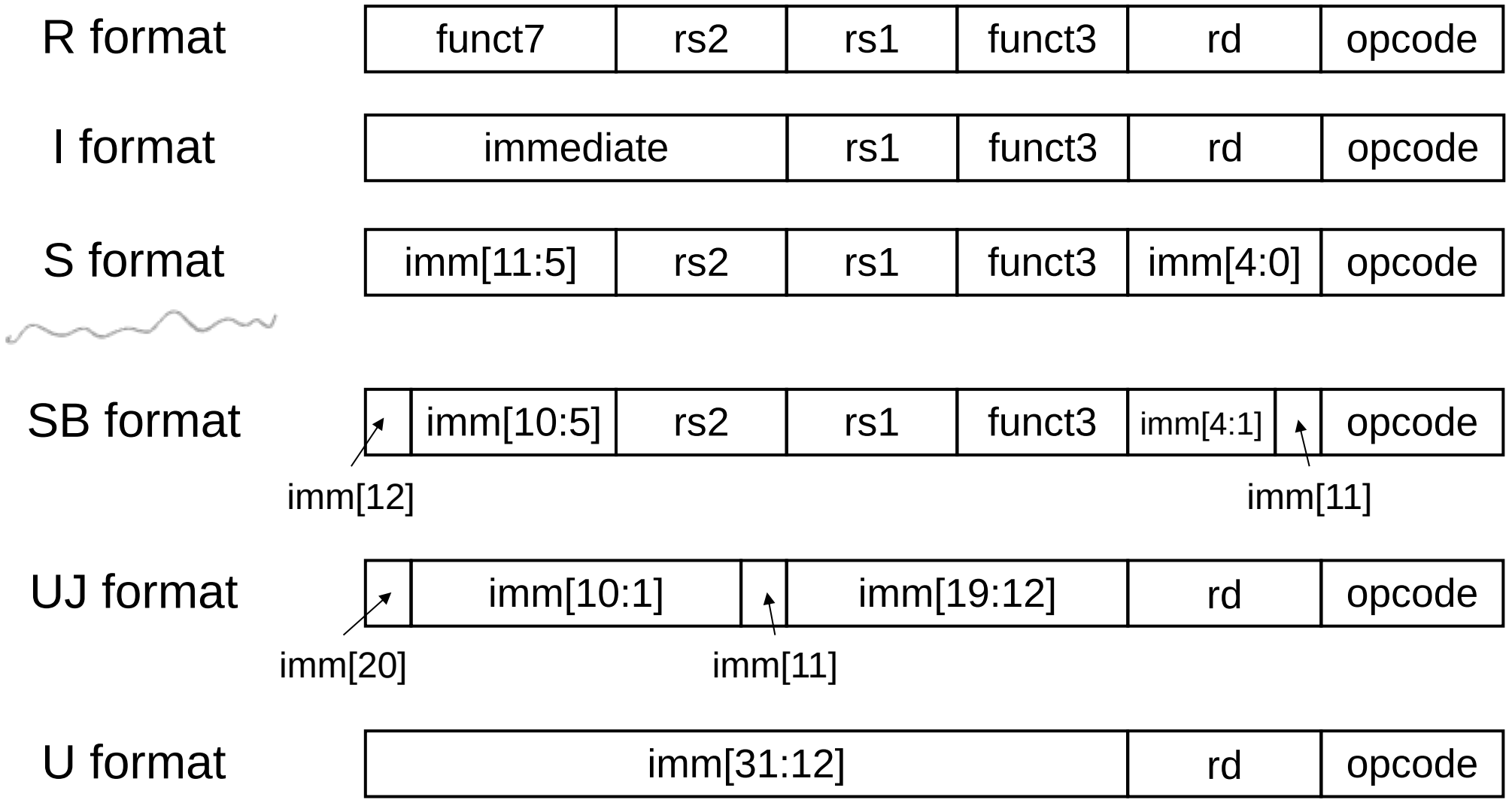
- Jump and link (`jal`) target uses a 20-bit immediate for a larger range

21 bit Immediate 2의 배수로 20bit
branch 13 bit Immediate 2의 배수로 12bit

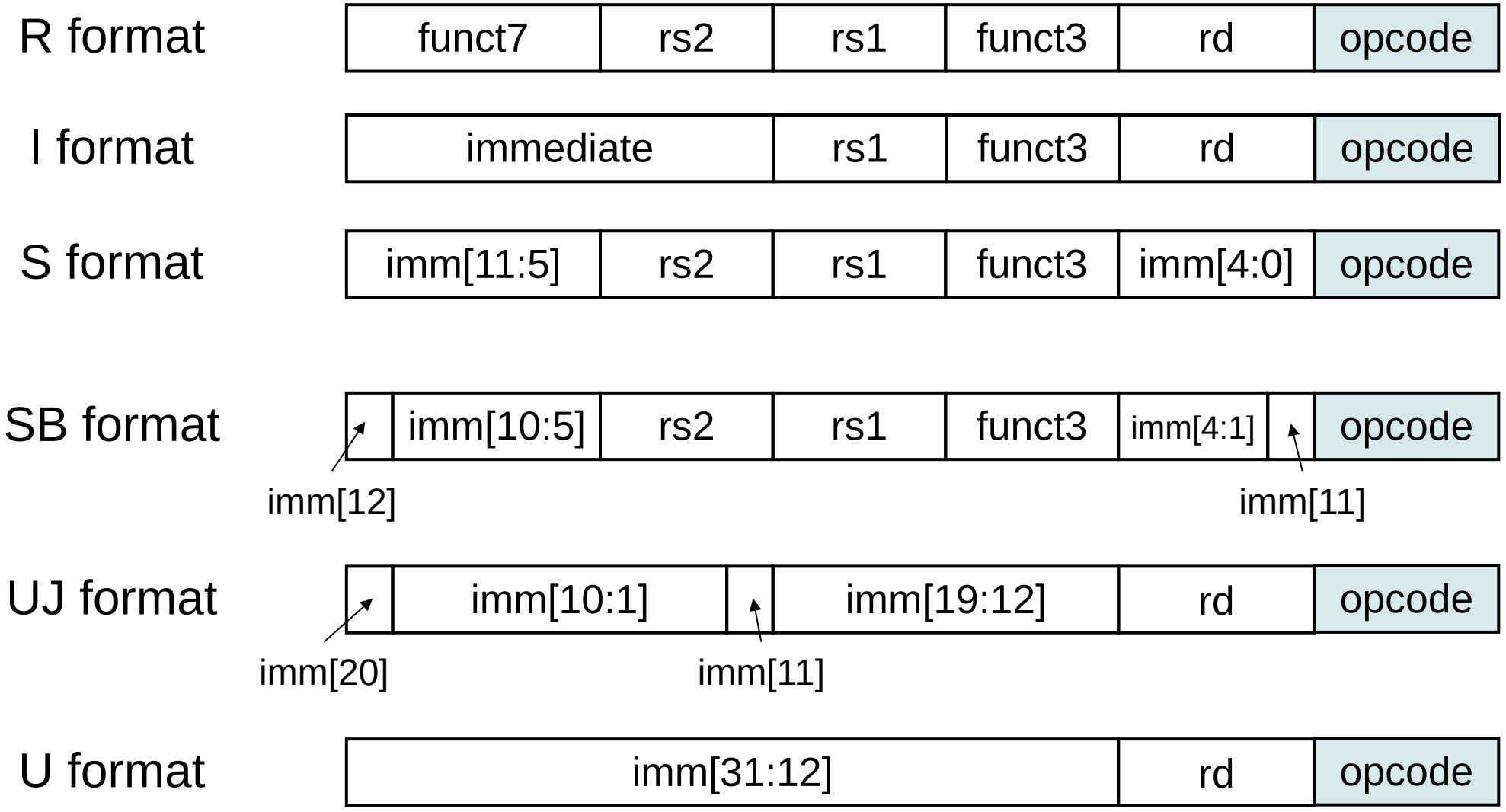


- For long jumps (e.g., to 32-bit absolute address),
  - `lui`: load address[31:12] to a temporary register
  - `jalr`: add address offset [11:0] to the temporary register and jump to target

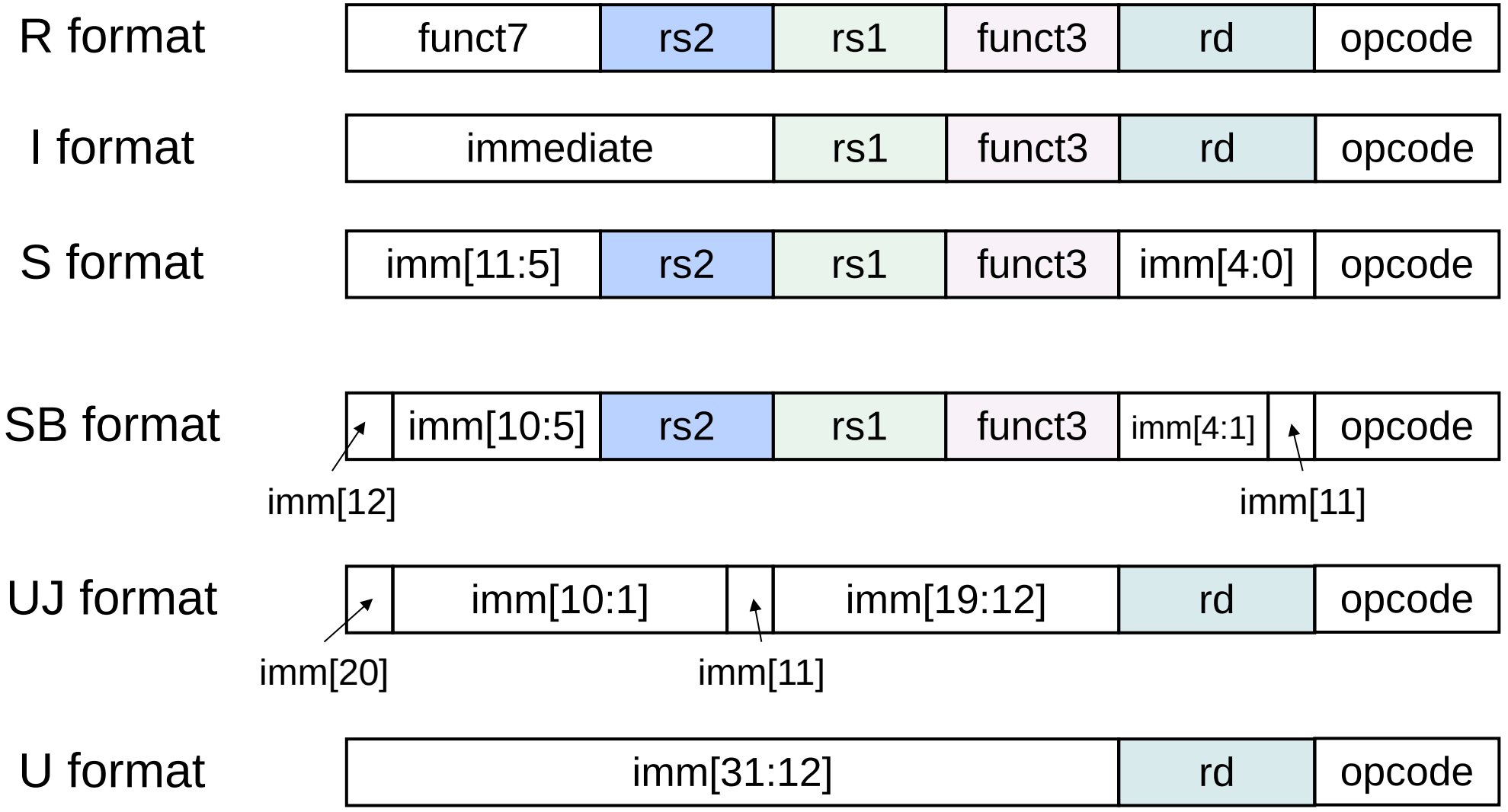
RISC-V Encoding Summary



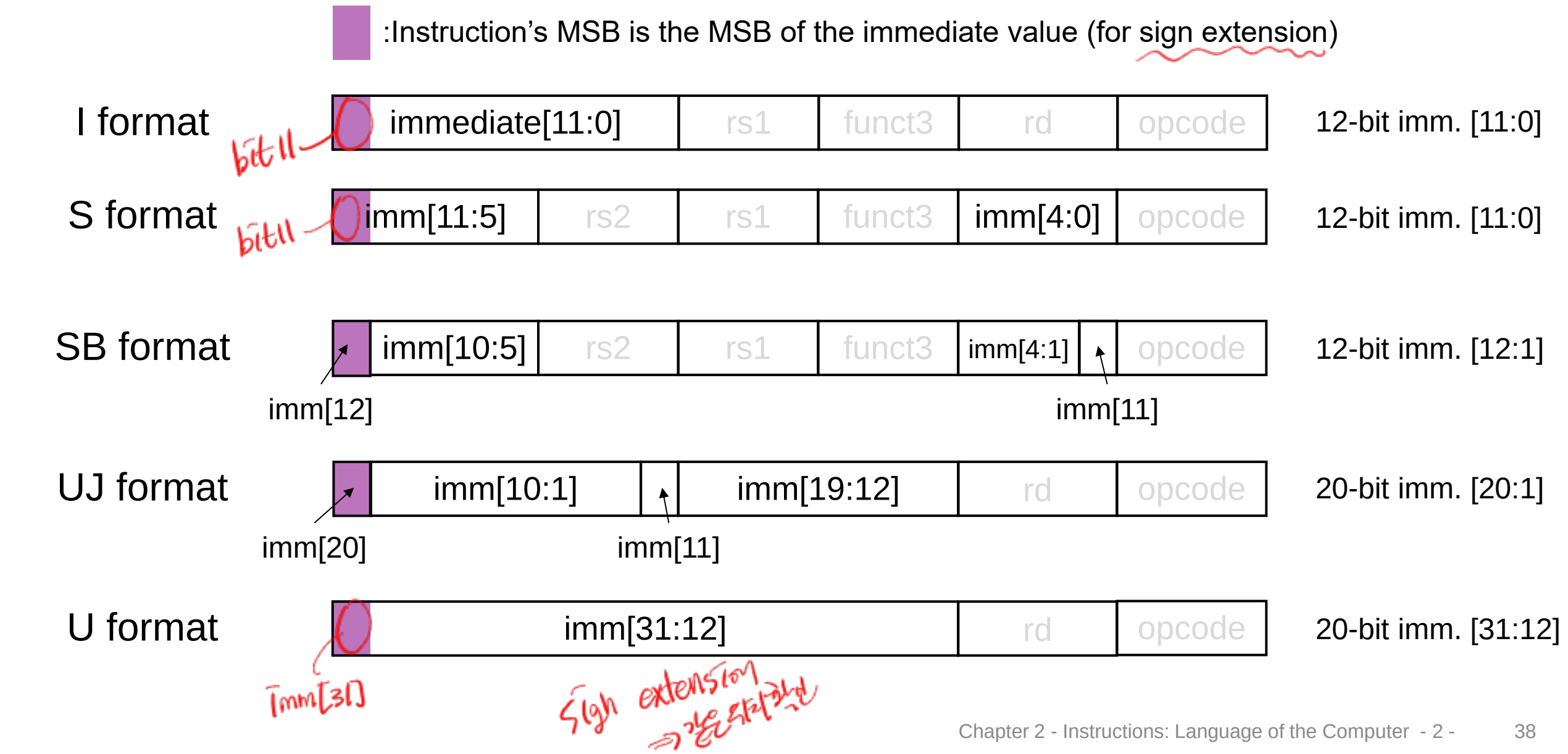
RISC-V Encoding Summary



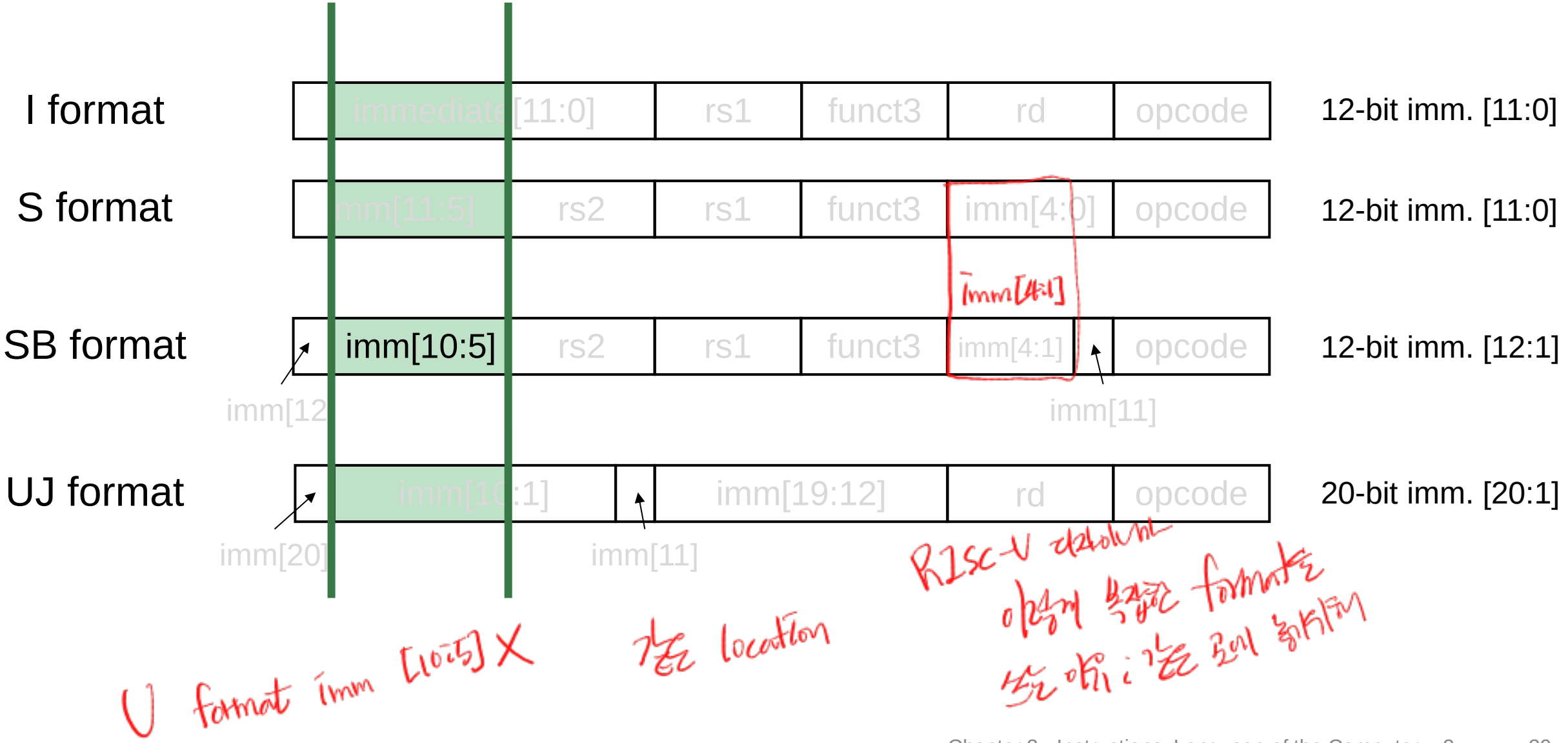
RISC-V Encoding Summary



RISC-V Encoding Summary



RISC-V Encoding Summary



Aligned vs. Non-aligned Immediate Values